

DESENVOLVIMENTO MOBILE

AULA 1 – PLATAFORMAS MÓVEIS, TYPESCRIPT E REACT NATIVE

CESAR School · 2026.2



VOCÊS JÁ SABEM CONSTRUIR UMA APLICAÇÃO COM TYPESCRIPT.

HOJE VAMOS LEVAR ISSO PARA DENTRO DO BOLSO DE BILHÕES DE PESSOAS.



AGENDA – 3 HORAS

Bloco	Tempo
Abertura: disciplina e projetos	15 min
1. Panorama das plataformas móveis	45 min
☕ Intervalo	10 min
2. TypeScript com olhos de mobile	35 min
3. Checkpoint conceitual	10 min
Fechamento	5 min



O PROJETO DA DISCIPLINA

Ao longo do semestre, cada estudante constrói **um** app:

- 🧘 **Rastreador de Micro-hábitos e Condicionamento Físico** – habit-tracker-expo

HOJE: TODO MUNDO CRIA O PROJETO EXPO QUE VAI USAR A PARTIR DA PRÓXIMA AULA.



AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

- 1 **Comparar** as 5 plataformas: Android, iOS, HarmonyOS, Tizen, webOS
- 2 **Justificar** nativo vs. cross-platform vs. web a partir de restrições reais
- 3 **Modelar** um domínio em TypeScript: props, estado, unions, generics

PARTE 1

PLATAFORMAS MÓVEIS

QUANTO VALE CADA PLATAFORMA?

STATCOUNTER – JUL/2026

mede tráfego web

	Mundo	Brasil
Android	68,4%	77,6%
iOS	31,6%	22,4%

COUNTERPOINT – Q1/2026

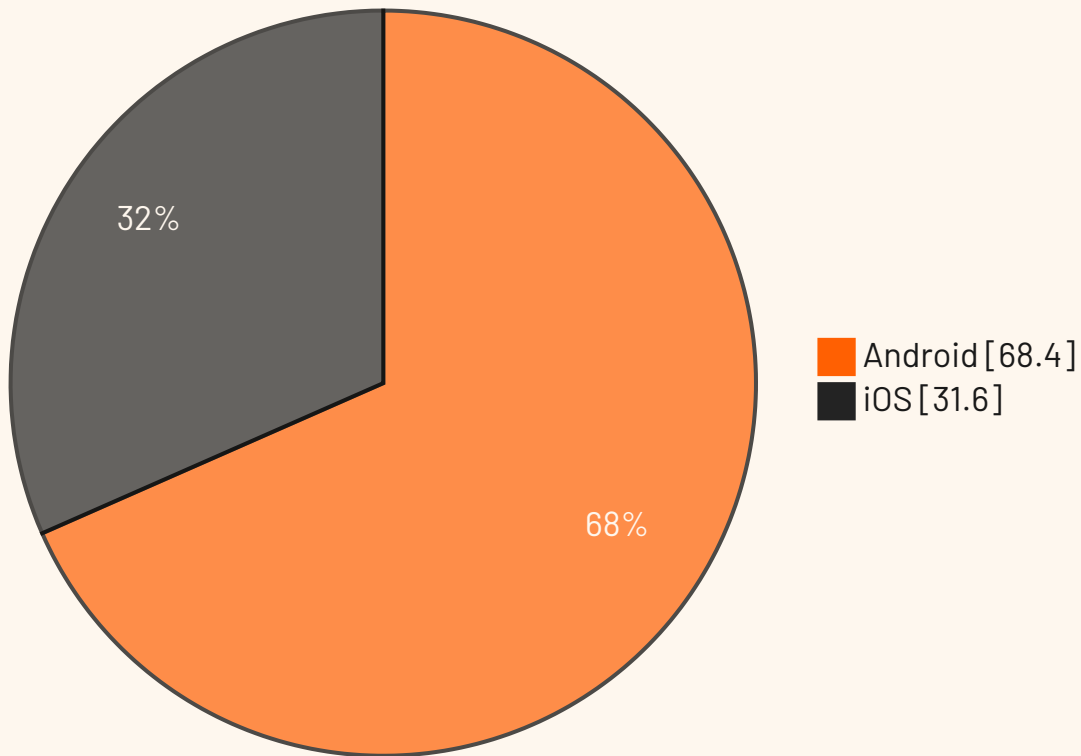
mede vendas de aparelhos

Android	iOS	HarmonyOS
~73%	~22%	~5%

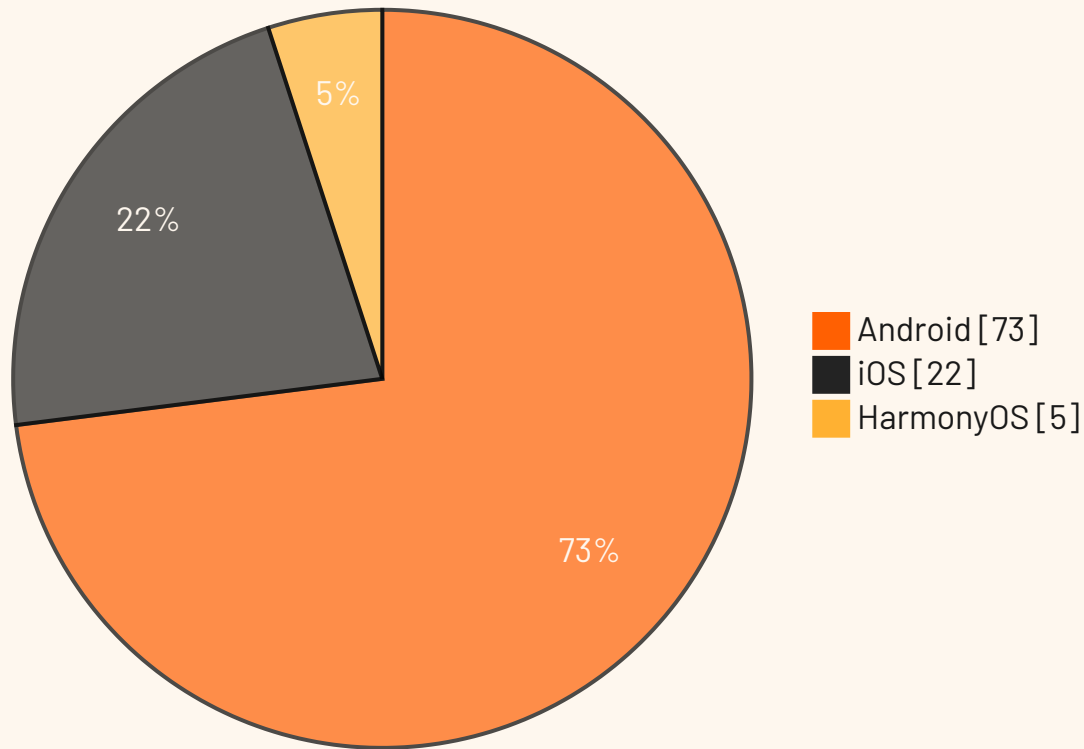


O MESMO MERCADO, DUAS FOTOGRAFIAS

StatCounter - trafego web



Counterpoint - vendas



**OS DOIS ESTÃO CERTOS.
POR QUE DISCORDAM?**

METODOLOGIA

- **Tráfego web** conta page views — quem navega mais, pesa mais
- **Vendas** contam aparelhos que saíram da loja neste trimestre
- **Base instalada** seria uma terceira coisa, diferente das duas

StatCounter não desagrega HarmonyOS.

Aparelhos Huawei são classificados como "Android" ou "outros" pelo user-agent. Uma plataforma com ~5% do mercado global fica **invisível**.

 **SAIBA O QUE O NÚMERO MEDE ANTES DE DECIDIR COM BASE NELE.**



ANDROID

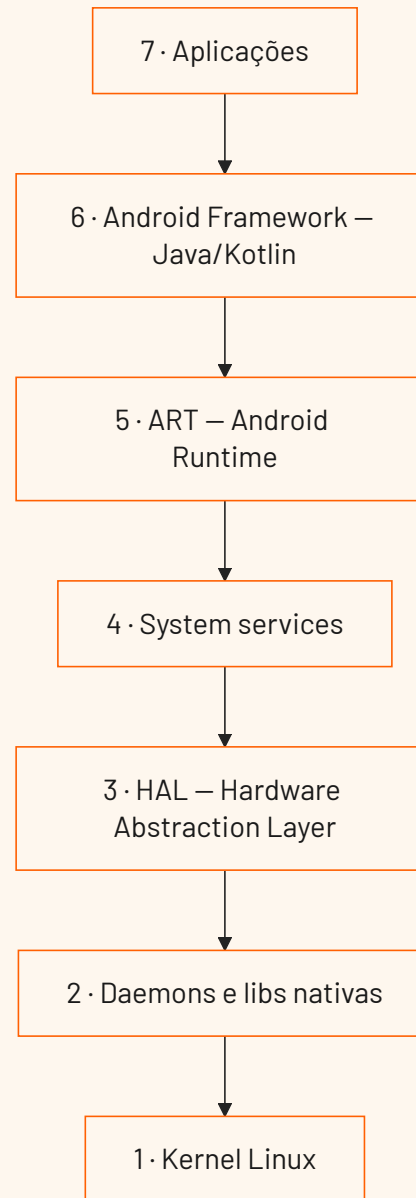
Dono	Google / AOSP — open source
Versão	17 "Cinnamon Bun" · API 37 (jun/2026)
Linguagem	Kotlin (Kotlin-first desde 2019)
UI	Jetpack Compose (declarativo)
IDE	Android Studio
Play Store	<code>targetSdk</code> ≥ 36 a partir de 31/ago/2026

DOR CARACTERÍSTICA: **FRAGMENTAÇÃO**

Muitos fabricantes · muitas versões em uso · muitos tamanhos de tela



ANDROID – ARQUITETURA (AOSP ATUAL: 7 CAMADAS)



⚠ Material antigo mostra **5** camadas. O AOSP hoje documenta **7**.

IOS

Dono	Apple — fechado , verticalmente integrado
Versão	26.6 (jul/2026) · iOS 27 em beta
Linguagem	Swift
UI	SwiftUI (UIKit segue suportado)
IDE	Xcode 26.6 — só roda em macOS
App Store	build com Xcode 26+ desde abr/2026

Vantagem: consistência — poucos aparelhos, atualização rápida

Restrição prática: sem Mac, sem desenvolvimento nativo iOS





SOBRE AQUELE DIAGRAMA DE 4 CAMADAS DO IOS

Você vai encontrar em todo lugar:

Cocoa Touch → Media → Core Services → Core OS

Ele vem de um documento da Apple que está **arquivado**. A documentação atual não organiza o iOS assim — descreve o núcleo Darwin/XNU e os frameworks por domínio.

Use como **modelo conceitual histórico**, não como arquitetura vigente.

OPENHARMONY / HARMONYOS

 **SÃO TRÊS COISAS DIFERENTES**

ESTE É O ERRO MAIS COMUM DO ASSUNTO.

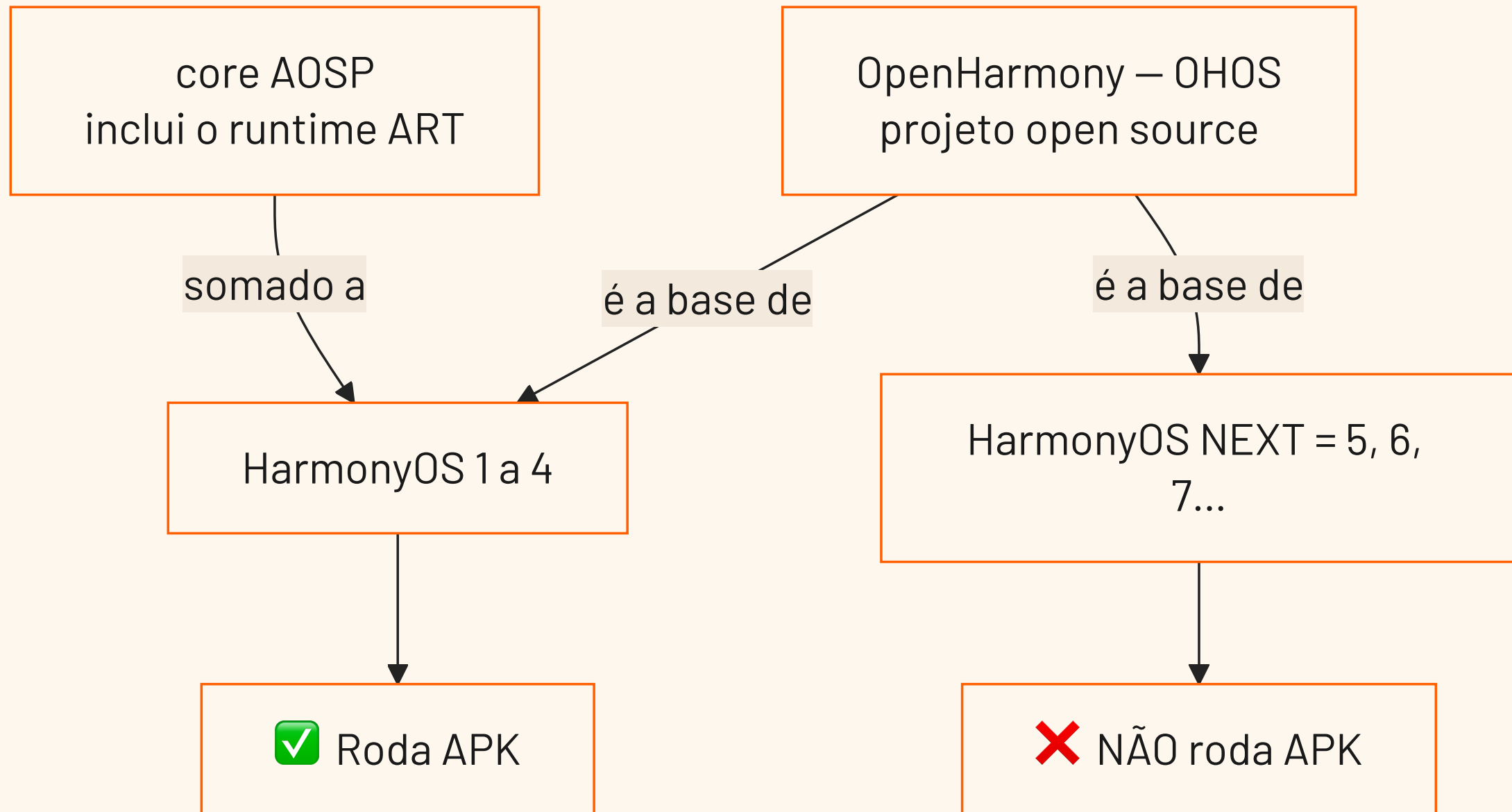
OS TRÊS NOMES

Nome	O que é	Roda APK?
OpenHarmony (OHOS)	Projeto open source , OpenAtom Foundation (2020) — a base técnica	—
HarmonyOS 1–4	Produto Huawei = OpenHarmony + core AOSP	✅ Sim
HarmonyOS NEXT = HarmonyOS 5 (out/2024) e superiores	Produto Huawei sem AOSP	❌ Não

A HUAWEI QUEBROU A COMPATIBILIDADE COM ANDROID DE PROPÓSITO.

Sem ART, sem camada de compatibilidade. Todo app é **reescrito**.

A DIFERENÇA ESTÁ EM UMA PEÇA SÓ





HARMONYOS – A STACK

Linguagem	ArkTS – superset do TypeScript
UI	ArkUI (declarativo)
Compilador	ArkCompiler (AOT)
IDE	DevEco Studio (gratuita)
Versões	HarmonyOS 6 · SDK 6.1.1 (API 24) · 7 em dev beta
OpenHarmony	6.1 LTS (mar/2026)

MERCADO

China, Q1/2026: ~19–20% — à frente do iOS pelo **7º trimestre seguido**
Global: ~5%





**A PLATAFORMA MÓVEL QUE CRESCE MAIS RÁPIDO NO MUNDO
ESCOLHEU TYPESCRIPT COMO BASE DA SUA LINGUAGEM
OFICIAL.**



TIZEN (SAMSUNG) – O QUE ELE NÃO É MAIS

✗ NÃO É O SISTEMA DOS CELULARES SAMSUNG

Celulares Samsung rodam **Android**.

✗ NÃO É MAIS O SISTEMA DOS SMARTWATCHES SAMSUNG

Desde o Galaxy Watch 4 (2021): **Wear OS**. Suporte aos relógios Tizen encerrado no fim de 2025.

TIZEN – ONDE ELE REALMENTE VIVE

SMART TVS · MONITORES · SIGNAGE · ELETRODOMÉSTICOS · IOT

Versão	Tizen 10.0 · SDK 10 (fev/2026)
Linguagem	Web – HTML/CSS/JS (engine V8)
Também	.NET/Xamarin, C/C++ nativo
Tooling	migrou de Java para Node.js + TypeScript , em VS Code

Uma versão maior por ano, junto com a linha de TVs.

WEBOS (LG)

Onde vive	TVs LG, monitores, projetores, signage, automotivo
webOS Hub	licenciado para 200+ marcas (Konka, Aiwa, Hyundai...)
Versão	webOS TV 26 · engine Chromium 132
Linguagem	Web – HTML/CSS/JS
Framework oficial	Enact – construído sobre React
Novidade	Flutter para webOS agora suportado
SDK	webOS Studio (VS Code), CLI, Simulator

COMPARATIVO

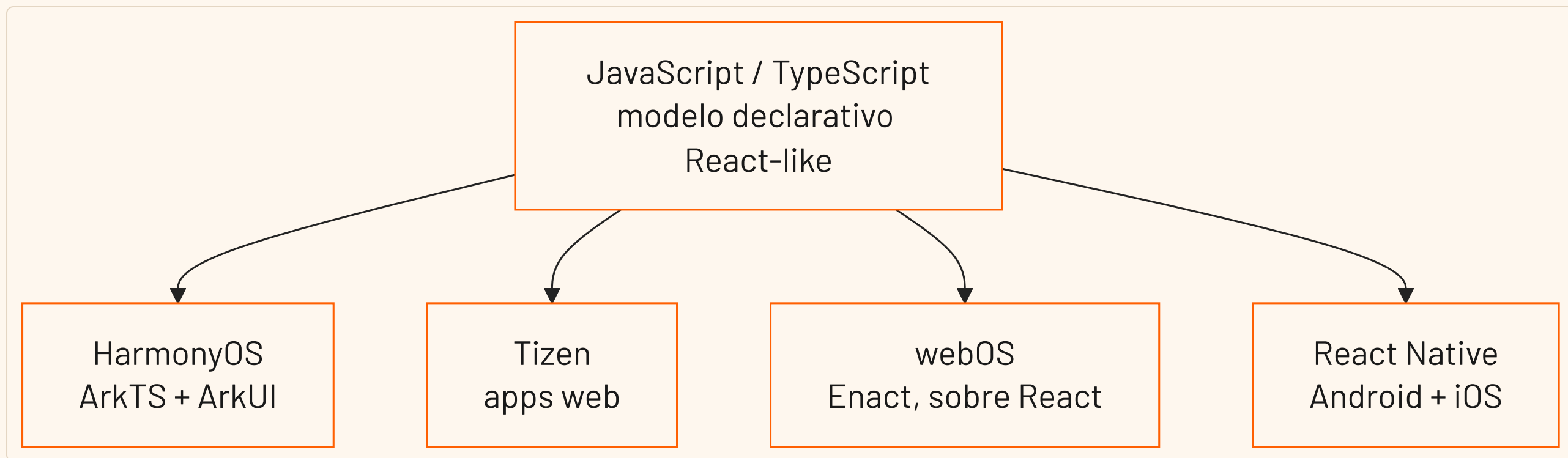
	Android	iOS	HarmonyOS	Tizen	webOS
Dono	Google	Apple	Huawei	Samsung	LG
Modelo	Aberto	Fechado	Misto	Aberto	OSE aberto
Linguagem	Kotlin	Swift	ArkTS	Web	Web
UI	Compose	SwiftUI	ArkUI	Web	Enact/React
Onde roda	Tudo	Apple	Celular, IoT	TV, IoT	TV, auto
Global	~68-73%	~22-32%	~5%	nicho	nicho
APK?	✓	✗	✗	✗	✗

3 DAS 5 PLATAFORMAS USAM JS/TS

HarmonyOS → ArkTS (superset de TS)
Tizen → apps web, tooling em TS
webOS → **Enact, sobre React**

NÃO É COINCIDÊNCIA. É A RAZÃO DESTA DISCIPLINA EXISTIR.

UMA LINGUAGEM, QUATRO DESTINOS





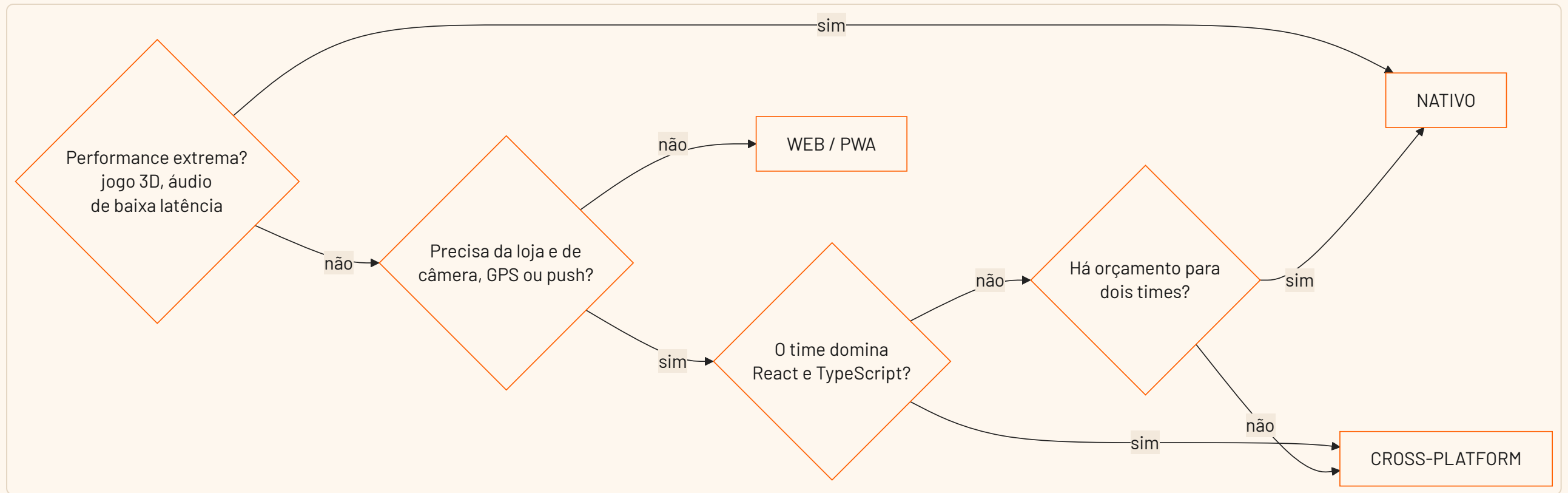
NATIVO, CROSS-PLATFORM OU WEB?

	Nativo	Cross-platform	Web/PWA
Performance	máxima	muito boa	limitada
Código	2 bases	1 base	1 base
Custo	alto	médio	baixo
Hardware	total	quase total	restrito
Loja	sim	sim	não
Quando	jogos, câmera, áudio de baixa latência	maioria dos apps de produto	alcance máximo, orçamento mínimo

ESCOLHA PELA RESTRIÇÃO MAIS DURA – NÃO PELA TECNOLOGIA MAIS LEGAL.



ÁRVORE DE DECISÃO



PARTE 2

TYPESCRIPT, COM OLHOS DE MOBILE

**VOCÊS JÁ USAM TS COM NODE, EXPRESS E NEXT.
A SINTAXE É A MESMA. O QUE MUDA É O PESO DE CADA RECURSO.**

POR QUE TS PESA MAIS NO MOBILE

NO WEB, UM BUG EM PRODUÇÃO:

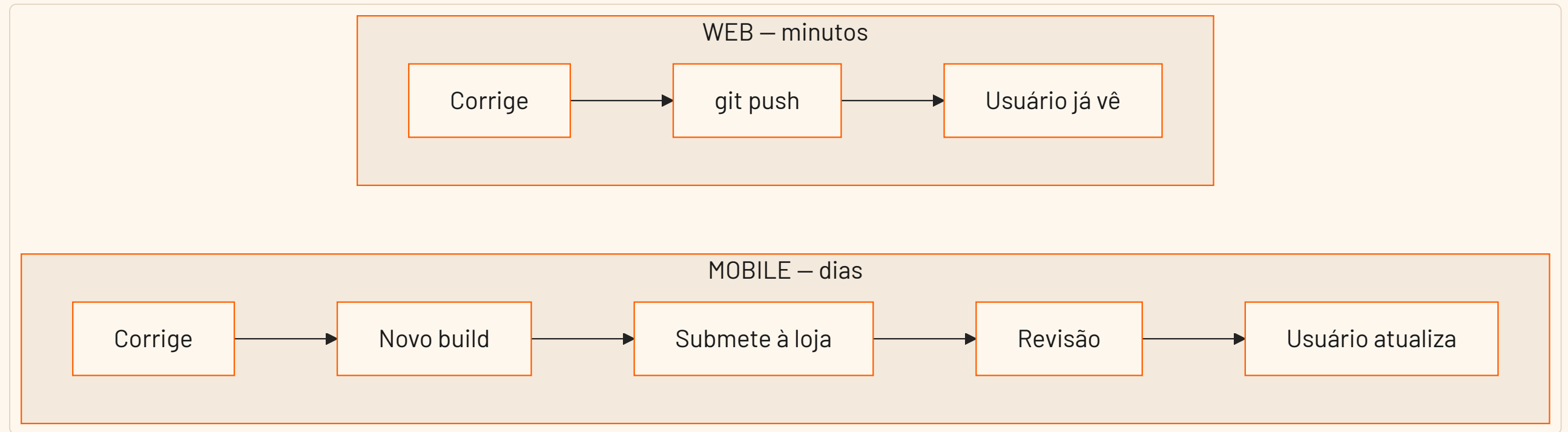
`git push` → resolvido em 10 minutos

NO MOBILE:

- 1 Corrige o código
- 2 Gera novo build
- 3 Submete para a loja
- 4 **Espera a revisão** (horas a dias)
- 5 Espera o usuário **atualizar o app**

O mesmo `undefined is not an object` custa **uma semana**.
E o usuário final não tem console para te contar o que aconteceu.

O CAMINHO DE UM BUG ATÉ O USUÁRIO



DUAS DESSAS ETAPAS NÃO DEPENDEM DE VOCÊ.

type VS interface

```
// interface – formas de objeto extensíveis
interface Usuario {
  id: string;
  nome: string;
}
interface Instrutor extends Usuario {
  especialidade: string;
}

// type – uniões, tuplas, funções, utilitários
type StatusHabitto = 'pendente' | 'concluido' | 'pulado';
type Duracao = [inicio: string, fim: string];
```

Regra prática: `interface` para objetos, `type` para o resto.
Não é regra técnica — é **consistência**.



ANTES DE VER A SOLUÇÃO: UMA ANALOGIA

Num restaurante existem dois jeitos de pedir:

- **Cardápio fechado** — você escolhe entre os pratos listados. Pediu algo que não existe? O garçom avisa **na hora**.
- **"Traga o que você tiver"** — o garçom aceita qualquer pedido e leva pra cozinha. Só lá — às vezes só quando chega errado à mesa — descobre-se que aquilo não existe.



Guarde essa imagem. Já voltamos a ela.

UNION LITERAIS – O RECURSO MAIS IMPORTANTE DE HOJE

```
// ✗ status: string – o "traga o que você tiver"
habito.status = 'conclído';      // faltou um "u"
habito.status = 'Pendente';     // maiúscula
habito.status = 'em espera';    // 🤖 valor que ninguém combinou

if (habito.status === 'finalizado') { /* nunca roda, ninguém avisa */ }
```

TODOS ESSES COMPILAM SEM RECLAMAR.

A SOLUÇÃO

```
type StatusHbito = 'pendente' | 'concluido' | 'pulado';

interface Hbito {
  status: StatusHbito;
}

habito.status = 'conclído'; // erro de compilação
```

ANALOGIA – O CARDÁPIO (RETOMANDO)

`string` = "traz o que eu pedir"

Union literal = **cardápio fechado** — só 3 pratos, e o garçom avisa **antes** da cozinha errar

BÔNUS: EXAUSTIVIDADE AUTOMÁTICA

```
function rotuloStatus(status: StatusHabit): string {  
  switch (status) {  
    case 'pendente': return 'Pendente';  
    case 'concluido': return 'Concluído';  
    case 'pulado': return 'Pulado';  
  }  
}
```

AMANHÃ ALGUÉM ADICIONA 'atrasado' AO TIPO...

→ **este switch** **passa a dar erro** até você tratar o novo caso.

O compilador vira uma lista de tarefas automática.

ESTADO DE TELA – O JEITO FRÁGIL

```
const [loading, setLoading] = useState(false);  
const [habito, setHabito] = useState<Habito | null>(null);  
const [erro, setErro] = useState<string | null>(null);
```

NADA IMPEDE O ESTADO IMPOSSÍVEL:

loading = true E erro = "falhou" E habito = {...}

O QUE A TELA MOSTRA?

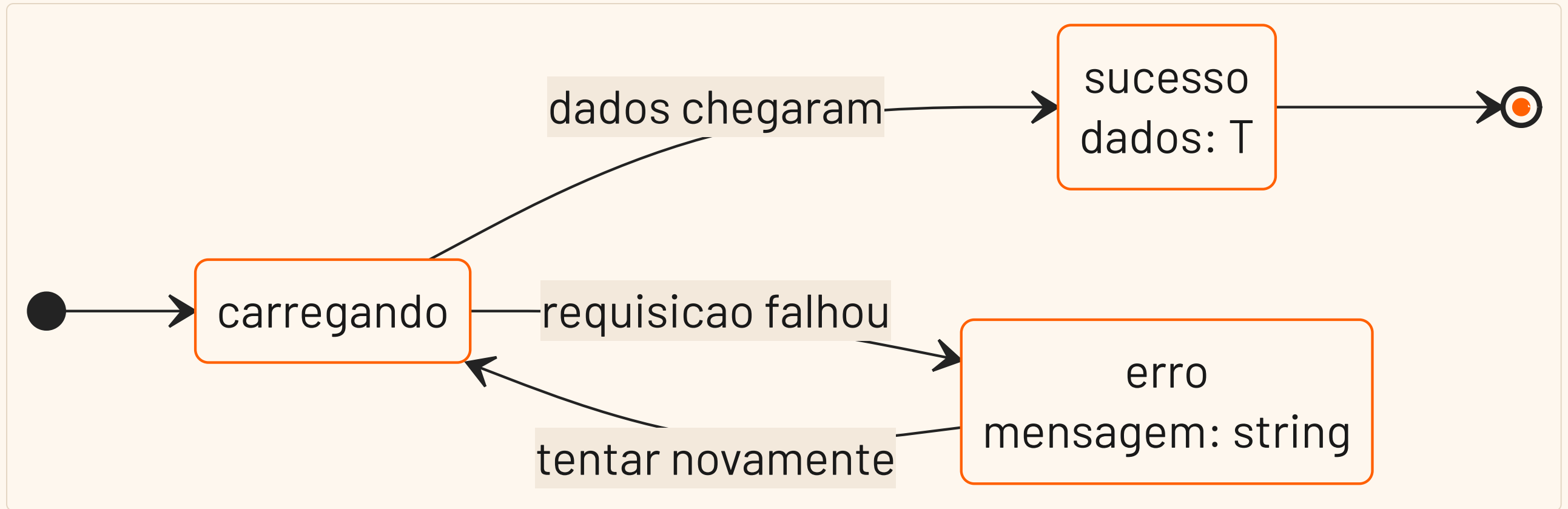
UNIÃO DISCRIMINADA – O JEITO ROBUSTO

```
type EstadoTela<T> =  
  | { tipo: 'carregando' }  
  | { tipo: 'sucesso'; dados: T }  
  | { tipo: 'erro'; mensagem: string };  
  
const [estado, setEstado] =  
  useState<EstadoTela<Habit>>({ tipo: 'carregando' });
```

```
switch (estado.tipo) {  
  case 'carregando': return <ActivityIndicator />;  
  case 'sucesso':    return <CardHabit habit={estado.dados} />; // ✓  
  case 'erro':       return <Erro texto={estado.mensagem} />;  // ✓  
}
```

Os estados impossíveis deixaram de ser **representáveis**.

TRÊS ESTADOS. EXATAMENTE TRÊS.



NÃO EXISTE SETA PARA "CARREGANDO E COM ERRO".

Aqui **T** é **um único** **Habito** – sem lista. Listas – **FlatList** e **SectionList** – serão vistas nas **próximas aulas**.

TIPANDO PROPS

```
type CardHabitProps = {
  titulo: string;
  status: StatusHabit;
  onPress: () => void;
  destacado?: boolean; // opcional
};

export function CardHabit({
  titulo, status, onPress,
  destacado = false, // default
}: CardHabitProps) {
  /* ... */
}
```

- `children: React.ReactNode` para componentes que envolvem outros
-  **React 19:** `ref` é prop normal — `forwardRef` não é mais necessário

GENERICOS: OS DOIS LUGARES ONDE VOCÊ VAI ENCONTRÁ-LOS

```
// 1. useState<T> – quando o valor inicial não revela o tipo
const [habito, setHabito] = useState<Habito | null>(null);
const [streak, setStreak] = useState(0); // aqui inferir basta

// 2. Funções que embrulham chamadas de API
async function buscarJson<T>(url: string): Promise<T> {
  const r = await fetch(url);
  if (!r.ok) throw new Error(`Falha: ${r.status}`);
  return r.json() as Promise<T>;
}

const habito = await buscarJson<Habito>('/habito-do-dia');
```

⚠ Aquele `as Promise<T>` é uma **promessa**, não uma verificação. (Voltaremos a isso na integração com API – Zod/Valibot)

UTILITY TYPES – UMA FONTE DE VERDADE

```
interface Habito {  
  id: string; titulo: string;  
  categoria: CategoriaHabito; frequencia: FrequenciaHabito;  
  status: StatusHabito; streakDias: number; criadoEm: string;  
}  
  
// o formulário não envia id/status/streakDias/criadoEm – o servidor gera  
type NovoHabito = Omit<Habito, 'id' | 'status' | 'streakDias' | 'criadoEm'>;  
  
// o card da tela só precisa de 4 campos  
type ResumoHabito = Pick<Habito, 'id' | 'titulo' | 'status' | 'categoria'>;  
  
type AtualizacaoHabito = Partial<NovoHabito>;
```

EXISTE UM TIPO. OS OUTROS SÃO DERIVADOS.

Adicione um campo à entidade → os derivados acompanham sozinhos.

O QUE EVITAR

```
// ❌ any desliga o compilador
function processar(dados: any) {
  return dados.qualquer.coisa; // compila, explode em runtime
}

// ❌ assertion: "confia em mim" – não gera verificação nenhuma
const habito = resposta as Habito;

// ✅ type guard: "eu verifiquei" – gera verificação de verdade
function ehHabito(v: unknown): v is Habito {
  return typeof v === 'object' && v !== null
    && 'id' in v && 'titulo' in v && 'status' in v;
}
```

any não é um tipo flexível. É **abrir mão da verificação**.

⚠️ AVISO DE AMBIENTE: TYPESCRIPT 7

JULHO/2026: COMPILADOR REESCRITO EM GO – 8 A 12× MAIS RÁPIDO

Type-check do VS Code: ~126s → ~11s

Mas: TS 7.0 ainda não expõe API programática.

`typescript-eslint` · `ts-jest` · `ts-morph` · Vue/Svelte/Astro → **não funcionam**

(API prevista para o TS 7.1)

👉 **NESTA DISCIPLINA: FIXE O TYPESCRIPT NA LINHA 6.X**

Lição de engenharia: uma versão pode ser tecnicamente superior e ainda **não estar pronta** para o seu projeto.

PARTE 3

CHECKPOINT

60 SEGUNDOS EM DUPLAS PARA CADA PERGUNTA



CHECKPOINT

- 1 Diferença entre **OpenHarmony**, **HarmonyOS 4** e **HarmonyOS NEXT**? Qual roda APK?
- 2 Startup, app de leitura de QR code, **2 devs que sabem TS**, prazo de **2 meses**. Nativo, cross-platform ou web? Justifique com **duas** restrições.
- 3 Por que `status: 'aberta' | 'em_analise' | ...` é melhor que `status: string`? **Dê um bug** que o primeiro previne.
- 4 **Tizen roda em celulares hoje?**

FECHAMENTO

Vocês já sabiam modelar uma entidade com TypeScript.
Hoje ela ganhou um destino. **O tipo não mudou – a plataforma mudou.**

PARA A PRÓXIMA AULA

- **Instalar:** Node.js **22.11+** e o app **Expo Go** no celular (o hands-on da Aula 2 começa com o projeto rodando no seu aparelho)
- **Ler:** Cap. 1 – História do Desenvolvimento do React Native · TypeScript Handbook – Narrowing
- **Fazer:** Exercícios 1 a 6 em `exercises.md` · Práticas 1 a 3 em `practice.md`

AULA 2 – REACT NATIVE: O QUE É, O QUE NÃO É, E O PROJETO RODANDO NO SEU CELULAR.